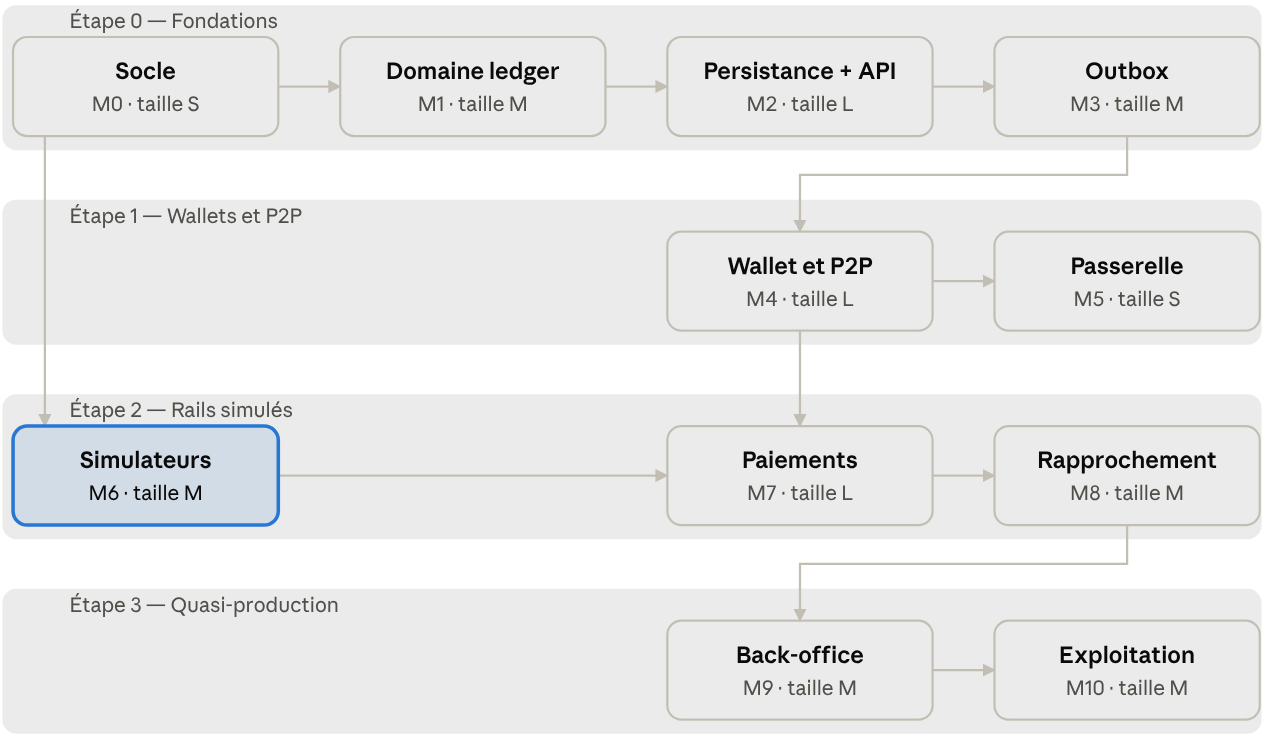
Plan de réalisation par jalons — Portefeuille UEMOA

Oct 28, 2026 · @SaifCores

Le système se construit en 11 jalons, de M0 à M10, répartis sur les quatre étapes du CDC ; chaque jalon se termine par des tests qui passent en CI, et le suivant commence seulement après.

Vue d'ensemble

Les simulateurs (M6) peuvent avancer en parallèle des jalons M1 à M5



11 jalons et leurs dépendances · tailles relatives S / M / L, estimées

Le chemin critique est M0 → M1 → M2 → M3 → M4 → M7 → M8 → M9 → M10 ; M5 (passerelle) et M6 (simulateurs) peuvent être faits à côté. Les tailles sont relatives (S < M < L) et servent à ordonner l'effort, pas à fixer des dates.

Jalon	Livrable principal	Exercices du CDC	Valide le jalon de la feuille de route
M0	Monorepo Gradle, CI, Compose, premier test ArchUnit	—	—
M1	Modèle comptable en Java pur, tests de propriété	1, 2	—
M2	Schéma PostgreSQL, algorithme de posting, API REST	3, 4, 5	—
M3	Outbox, RabbitMQ, contrats d'événements	—	Invariant prouvé
M4	Contexte wallet : KYC, plafonds, frais, P2P idempotent	6	—
M5	Traefik, Keycloak, jetons de service	—	Zéro découvert
M6	Simulateurs PI-SPI, opérateurs, banques, pannes	—	—
M7	Contexte paiements : sagas, inbox, enquêtes de statut	7, 8	—
M8	Contexte rapprochement, couverture, dérive	9, 10, 11	30 jours sans écart
M9	Back-office, double validation, AML, rapports	12	—
M10	Observabilité, comptes asynchrones, charge, reprise	13, (14)	Critères d'acceptation du CDC

Étape 0 — Fondations

Ces quatre jalons livrent un ledger correct, persistant et observable par événements ; rien d'autre ne commence tant que l'invariant n'est pas prouvé.

M0 — Socle du monorepo · taille S

Objectif : un dépôt qui compile, teste et démarre une base en une commande.

- [ ] Créer le monorepo `senwallet/` avec `settings.gradle.kts`, `gradle/libs.versions.toml` (Java 25, Spring Boot 4.x, jqwik, Testcontainers, ArchUnit).
- [ ] `build-logic/` : plugins de convention `senwallet.java-library` (sans Spring) et `senwallet.spring-service`.
- [ ] Module `platform/shared-kernel` : `Money`, identifiants typés (`AccountId`, `WalletId`…), `DomainEvent`.
- [ ] Squelette des modules `contexts/ledger/{domain, application, infrastructure, boot}` vides.
- [ ] `deploy/docker-compose.yml` avec PostgreSQL 16 seul.
- [ ] CI (GitHub Actions) : build, tests, rapport de couverture.
- [ ] Premier test ArchUnit : le package `..domain..` ne dépend ni de Spring ni de Jackson.

Terminé quand `./gradlew build` passe en CI et `docker compose up` démarre la base.

M1 — Domaine du ledger · taille M

Objectif : tout le modèle comptable en Java pur, prouvé par des tests de propriété sans base de données.

- [ ] Objets valeur : `Amount` (> 0, XOF), `Direction`, `NormalBalance`, `ExternalId`, `TransactionStatus`.
- [ ] Agrégat `LedgerAccount` avec ses quatre seaux et le calcul posted / pending / available pour les deux normalités.
- [ ] Agrégat `LedgerTransaction` : refus si moins de deux écritures ou si débits ≠ crédits ; transitions pending → posted / archived seulement.
- [ ] `BalanceCondition` (gt, gte, eq, lte, lt sur posted / pending / available).
- [ ] `PostingService` : applique une transaction aux comptes et évalue les conditions.
- [ ] Événements `TransactionPosted`, `TransactionPending`, `TransactionArchived`.
- [ ] Ports `LedgerAccountRepository`, `LedgerTransactionRepository` (interfaces) + implémentations en mémoire pour les tests.

Terminé quand le test jqwik de 10 000 transactions aléatoires garde Σ débits = Σ crédits (exercice 1) et le scénario SendCash donne les soldes attendus en mémoire (exercice 2).

M2 — Persistance et API du ledger · taille L

Objectif : l'algorithme de posting de l'architecture (section 4) en PostgreSQL, exposé en REST.

- [ ] Migration Flyway `V1__ledger_schema.sql` : tables, enums, index, partitions mensuelles des écritures.
- [ ] Garde-fous en base : trigger d'équilibre différé, trigger d'immuabilité, rôle sans `UPDATE` / `DELETE` sur les écritures.
- [ ] Adaptateurs `JdbcClient` : insertion idempotente (`ON CONFLICT`), verrous `FOR UPDATE` triés, mise à jour des compteurs.
- [ ] `PostTransactionHandler` et `TransitionHandler` (pending → posted / archived), endpoint de reversal.
- [ ] API REST : `/ledgers`, `/ledger_accounts`, `/ledger_transactions`, `/balances?as_of=`, erreurs en `ProblemDetail` (409, 422).
- [ ] Contrat OpenAPI dans `contracts/openapi/ledger.yaml`.
- [ ] Tests d'intégration Testcontainers.

Terminé quand les exercices 3, 4 et 5 passent contre une vraie base : soldes identiques depuis les compteurs et depuis les écritures, 1 000 rejeux = une transaction, 100 débits concurrents de 20 000 sur 1 000 000 XOF = exactement 50 succès.

M3 — Outbox et événements · taille M

Objectif : chaque posting publie un événement, sans perte ni doublon d'effet.

- [ ] Module `platform/event-contracts` : enveloppe commune et records des événements ledger, schémas JSON.
- [ ] Table `outbox` écrite dans la transaction de posting.
- [ ] `OutboxRelay` planifié (`FOR UPDATE SKIP LOCKED`), publication RabbitMQ avec confirmations.
- [ ] RabbitMQ dans Docker Compose, exchange topic `ledger`.
- [ ] Bibliothèque de consommation idempotente (`processed_events`) dans `platform/`.
- [ ] Test : arrêter le relais au milieu d'un lot, redémarrer, vérifier qu'aucun événement n'est perdu et qu'un consommateur de test n'applique chaque événement qu'une fois.

Terminé quand le jalon « Invariant prouvé » de la feuille de route est atteint : M1 à M3 verts en CI.

Étape 1 — Wallets et P2P

Ces deux jalons ajoutent les utilisateurs, les plafonds et les transferts internes, derrière une passerelle sécurisée ; aucun rail externe n'est encore branché.

M4 — Contexte wallet et P2P · taille L

Objectif : un utilisateur inscrit et vérifié peut envoyer des XOF à un autre, avec frais et plafonds appliqués par le ledger.

- [ ] Modules `contexts/wallet/{domain, application, infrastructure, boot}` et base `wallet`.
- [ ] Agrégats `Customer` (téléphone, niveau KYC simulé), `Wallet` (statut, `ledger_account_id`), `Transfer`.
- [ ] Configuration datée (`valid_from`) : `LimitPolicy` (niveaux N1, N2, marchand ; plafond 4 000 000 XOF) et `FeeSchedule` (P2P 1 %).
- [ ] Création atomique du wallet : compte ledger + alias PI-SPI enregistré localement ; annulation si une étape échoue.
- [ ] Client HTTP du ledger généré depuis `contracts/openapi/ledger.yaml`, avec délais et rejeux Resilience4j.
- [ ] `POST /transfers` : `external_id` = `p2p:{transfer_id}`, condition `available >= 0` sur l'émetteur, `pending <= plafond` sur le bénéficiaire, frais vers le compte revenus.
- [ ] Table `idempotency_keys` et filtre `Idempotency-Key` (rejeu = même réponse, corps différent = 409).
- [ ] Consommation des événements ledger pour l'historique et les notifications (SMS simulés en log).

Terminé quand l'exercice 6 passe (crédit au-delà du plafond rejeté en 422 sans écriture) et que la partie interne de SendCash (P2P 50 000 + 500 de frais) s'exécute via l'API wallet.

M5 — Passerelle et sécurité · taille S

Objectif : un seul point d'entrée authentifié ; le ledger devient invisible de l'extérieur.

- [ ] Keycloak dans Docker Compose : realm `senwallet`, utilisateurs de test, rôles `customer`, `ops`, `finance`, `compliance`, `approver`.
- [ ] Traefik : TLS local, routage vers wallet (et paiements au jalon M7), quota par client.
- [ ] Services en OAuth2 Resource Server ; jetons de service (client credentials) pour wallet → ledger.

- [ ] Le ledger refuse tout appel sans jeton de service autorisé.
- [ ] Rôles PostgreSQL distincts par service.

Terminé quand le jalon « Zéro découvert » est atteint : exercices 4 à 6 verts à travers la passerelle, et un appel direct au ledger depuis l'extérieur échoue.

Étape 2 — Rails simulés

Ces trois jalons branchent l'argent « réel » simulé : simulateurs avec pannes, sagas de paiement, puis rapprochement de tout ce qui entre et sort.

M6 — Simulateurs · taille M

Objectif : imiter PI-SPI, deux opérateurs et deux banques, avec des pannes déclenchables par API. Peut démarrer dès M0, en parallèle de l'étape 1.

- [ ] `simulators/pispi` : virements sortants, simulation d'entrants (`/control/inbound`), webhook vers la plateforme, rapport de règlement.
- [ ] `simulators/mno` : collectes (top-up), décaissements (cash-out), float, enquête de statut, callbacks, fichier de règlement ; deux instances `mno-wave` et `mno-orange` .
- [ ] `simulators/bank` : relevés CSV des comptes de cantonnement déposés dans MinIO ; deux instances.
- [ ] API `/control/faults` commune avec les modes `reject` , `latency` , `timeout_then_success` , `timeout_then_failure` , `duplicate_callback` , `out_of_order_callback` , `float_exhausted` , `missing_statement_line` , `wrong_amount` .
- [ ] Signature HMAC-SHA256 des callbacks avec horodatage.
- [ ] Les simulateurs et MinIO ajoutés à Docker Compose.

Terminé quand chaque mode de panne est déclenchable par un test et observable dans la réponse ou le callback du simulateur.

M7 — Contexte paiements et sagas · taille L

Objectif : top-up, cash-out et PI-SPI fonctionnent de bout en bout, y compris quand les rails expirent ou se répètent.

- [ ] Modules `contexts/payments/{domain, application, infrastructure, boot}` et base `payments` .

- [ ] Agrégat `PaymentOrder` : machine à états du cash-out (créé, blocage posé, soumis, terminé, échoué, rejeté, inconnu, revue manuelle) ; transitions qui ne reculent jamais.
- [ ] Port `RailGateway` et adaptateurs anticorruption `PispiGateway`, `MobileMoneyGateway`.
- [ ] Sagas cash-out, top-up (contrôle du cumul mensuel de 15 000 000 XOF), PI-SPI sortant ; PI-SPI entrant posté directement, ou en compte d'attente si l'alias est inconnu.
- [ ] Inbox des callbacks (`provider`, `provider_event_id`) et worker de traitement.
- [ ] Worker d'enquêtes de statut (30 s, 1, 2, 5, 10 min) puis revue manuelle.
- [ ] Endpoints `POST /topups`, `POST /withdrawals` exposés par la passerelle.

Terminé quand les exercices 7 et 8 passent, ainsi que SendCash complet (dépôt PI-SPI, P2P, cash-out Orange) et Pay2Day (1 500 000 répartis sur 100 livreurs, retraits Wave).

M8 — Rapprochement · taille M

Objectif : chaque franc entré ou sorti est rapproché, et la couverture du cantonnement est calculée chaque jour.

- [ ] Modules `contexts/reconciliation/{domain, application, infrastructure, boot}` et base `recon`.
- [ ] Agrégats `ReconciliationRun`, `Match`, `Break`, `CoverageReport`.
- [ ] Jobs `recon-bank`, `recon-pispi`, `recon-mno` avec les règles de rapprochement (identifiant du rail, puis `external_id`, puis approché avec revue).
- [ ] Job `coverage` : couverture ≥ 100 % et part à vue ≥ 65 % ; événement `coverage.breach` qui gèle les émissions et les retraits.
- [ ] Dans le service ledger : `DriftScanner` (nocturne + échantillon) et reconstruction des compteurs.
- [ ] Horloge simulée pour rejouer 30 jours d'activité en quelques minutes.

Terminé quand le jalon « 30 jours sans écart » est atteint : exercices 9, 10 et 11 verts, et 30 jours simulés sans écart inexpliqué.

Étape 3 — Quasi-production

Ces deux jalons rendent le système exploitable : des humains peuvent intervenir sous contrôle, et l'on mesure qu'il tient la charge et la panne.

M9 — Back-office et conformité · taille M

Objectif : ops, finance et conformité travaillent depuis une interface, avec double validation sur tout ce qui touche à l'argent.

- [ ] Application `backoffice/` (React ou équivalent) connectée à Keycloak.
- [ ] Recherche par ID, téléphone, `external_id` ou métadonnées ; vue d'un wallet et de son historique.
- [ ] Ajustement manuel : demande par un utilisateur, validation par un second avec le rôle `approver`, puis seulement posting au ledger.
- [ ] Gel / dégel (sorties seules ou tout) ; traitement des ordres en revue manuelle.
- [ ] Règles AML simples dans le module conformité : vélocité, fractionnement sous les plafonds, comptes mules ; file d'alertes.
- [ ] Tableaux des écarts de rapprochement et de la couverture ; rapports e-money et déclarations CENTIF générés (non transmis).
- [ ] Journal d'audit chaîné par hachage.

Terminé quand l'exercice 12 passe : un ajustement sans second validateur est refusé, et la piste d'audit est complète.

M10 — Observabilité, charge et reprise · taille M

Objectif : prouver que le système tient 10× le pic, se voit de l'intérieur et se restaure sans perte.

- [ ] OpenTelemetry (Micrometer Tracing) sur tous les services ; `trace_id` copié dans les métadonnées ledger et les événements.
- [ ] Prometheus, Grafana, Loki, Tempo dans Docker Compose ; tableaux de bord postings, rejets, sagas, simulateurs, couverture.
- [ ] Alertes : couverture < 100 %, dérive > 0, outbox en retard > 1 min, ordre inconnu > 30 min, float bas.
- [ ] Comptes asynchrones (revenus, compensation, cantonnement) et `BalanceAggregator`.
- [ ] Scripts k6 dans `tests/load/` : rafales P2P, top-ups, cash-outs, comptes chauds.
- [ ] Exercice de reprise : sauvegarde, destruction de la base ledger, restauration, rapprochement.
- [ ] Optionnel : exercice 14, refaire le ledger sur un moteur open source et faire passer les mêmes tests.

Terminé quand l'exercice 13 passe (latence p95 du posting ⩽ 300 ms à 10× le pic, aucune erreur d'équilibre) et que tous les critères d'acceptation du CDC (section 14.3) sont cochés.

Règles de travail communes

Un jalon n'est terminé que lorsque ses tests passent en CI, pas lorsque son code est écrit.

Définition de terminé (chaque jalon)

- Tous les tests du jalon et des jalons précédents sont verts en CI ; les tests ArchUnit aussi.
- Contrats OpenAPI et schémas d'événements à jour dans `contracts/` et `platform/event-contracts`.
- Migrations Flyway versionnées, jamais modifiées après fusion.
- `docker compose up` démarre tout ce que le jalon a ajouté.
- Un court ADR est ajouté au dossier d'architecture pour chaque décision prise en cours de route.
- Une note de démo dans `docs/milestones/Mx.md` : ce qui a été construit, ce qui a été appris, ce qui reste.

Branches et CI

- Une branche par jalon (`milestone/m2-ledger-persistence`), fusionnée par pull request quand la définition de terminé est remplie.
- Pipeline : build → tests unitaires et propriétés → ArchUnit → tests d'intégration Testcontainers → (à partir de M10) test de charge court.
- Un tag Git par jalon (`m0` … `m10`) pour pouvoir revenir à chaque étape.

Documents de référence : CDC de l'étude de cas · Dossier d'architecture